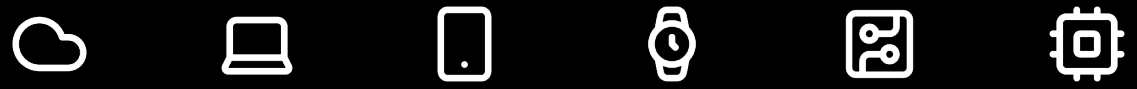


# APIs & Observability

Lecture 6: packages, feature flags, GraphQL & REST, Crashlytics

Dinu-Ștefan Rusu

ORIENTATION

# Who's teaching this course


## Dinu-Ștefan Rusu

dinu_stefan.rusu@upb.ro
Microsoft Teams course channel
rusudinu.com


45+

Flutter apps shipped

400k

installs across them

Appeared in, spoken at & partnered with

TODAY

# What you should leave with

| | |
|---|---|
| **Add a package safely** | Read a pub.dev page, pin a constraint, and know what pubspec.lock is actually for |
| **Ship behind a flag** | Firebase Remote Config: defaults, fetch and activate, plus staged rollout and a kill switch |
| **Compare REST and GraphQL** | Over- and under-fetching, caching, typing, N+1, and when each one wins |
| **See your app in production** | Events, p95 latency, crash-free users, and which alerts are worth firing |

PART 1 · PACKAGES

# Depending on someone else's code

pub.dev, version constraints, and the lockfile you must commit

PACKAGES

# Choosing a package you can live with

### pub.dev is the registry

Every Dart and Flutter package lives here. Each page carries the API docs, a runnable example, and the exact install command.

### Read the scores critically

Likes and downloads are popularity. Pub points are mechanical: docs, formatting and platform support. They are not a verdict on quality.

### Check the last publish date

A plugin last released against Flutter 3.19 is maintenance you are taking on yourself.

### Check the platforms

The page lists Android · iOS · web · desktop. A plugin with native code often supports fewer of them than you assume.

### Open the dependency tree

You adopt every transitive dependency too. Two packages that pin conflicting versions is the classic unresolvable pubspec.

### Check the license

MIT, BSD and Apache-2.0 are fine for your project. Anything copyleft is a conversation to have before you ship, not after.

---

A dependency is quick to add and slow to remove, so spend a minute checking it first.

# Adding a package

```
# pubspec.yaml
dependencies:
  flutter:
    sdk: flutter
  http: ^1.5.0
  firebase_core: ^4.1.0
  firebase_crashlytics: ^5.0.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  build_runner: ^2.5.0
```

```
# Writes the constraint AND resolves it
flutter pub add http
flutter pub add dev:build_runner
# Download exactly what the lockfile pins
flutter pub get
# Newest inside your constraints
flutter pub upgrade
# Raise the constraints themselves
flutter pub upgrade --major-versions
# What has moved on without you?
flutter pub outdated
```

**Do not hand-edit pubspec.yaml to add a package.** flutter pub add writes the current constraint and resolves in one step; the versions above are illustrative, and will be stale by the time you read them.

# Version constraints and the lockfile

| | What it allows | What it blocks |
|---|---|---|
| ^1.5.0 | >= 1.5.0 and < 2.0.0 | 2.0.0, a breaking major |
| ^0.3.2 | >= 0.3.2 and < 0.4.0 | 0.4.0; before 1.0, minors may break |
| 1.5.0 | exactly 1.5.0, forever | every patch, including security fixes |
| any | whatever the resolver picks | nothing at all; never ship this |

**pubspec.yaml says what you will accept. pubspec.lock records what you actually got.**

Commit pubspec.lock for an application: it is what makes your build, your teammate's build and CI resolve to identical code.

Do not commit it for a package you publish: there you want consumers to resolve against their own constraint set.

flutter pub get obeys the lockfile. flutter pub upgrade rewrites it. That is the whole difference between the two commands.

PART 2 · FEATURE FLAGS

# Deploy is not release

Firebase Remote Config: staged rollout, kill switch, and flag debt

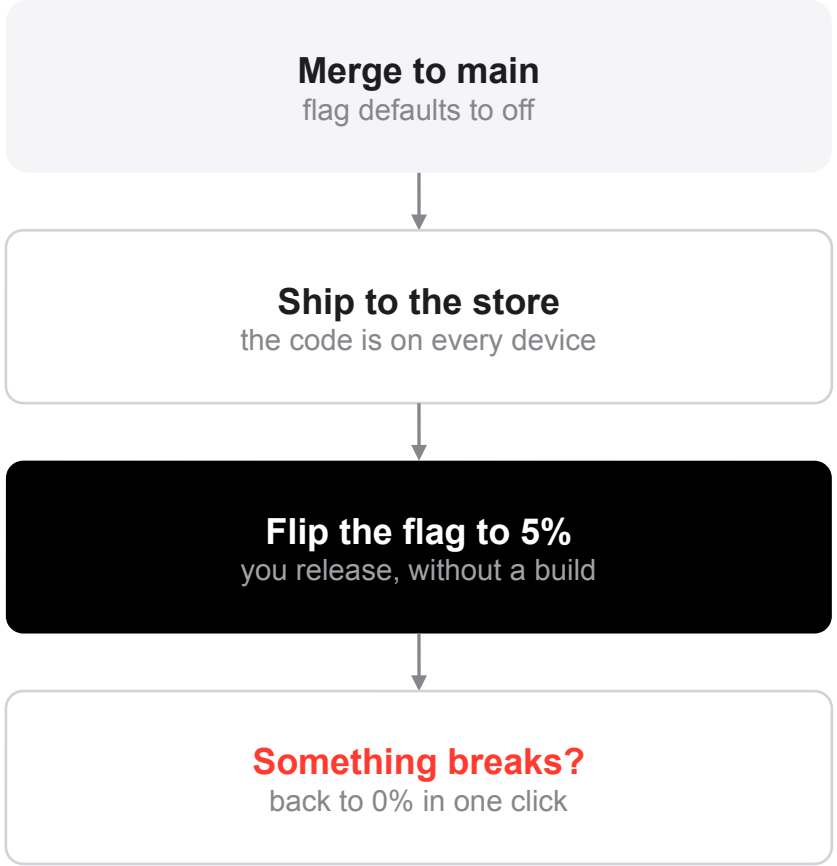
# Deploy is not release

A staged rollout ships a binary to 1% of users, then 5, 10, 50, 100. The store does this for you, and you can halt it if crashes spike. The store runs the rollout; your job is to watch the crash rate while it climbs.

But a rollout only controls who gets the binary, and users who already have it keep it. You cannot take code back, and a fix takes another review cycle to reach them.

A feature flag separates the two: the code ships to everyone, switched off. A server decides, per user and at runtime, whether it is on.

**Deploy is a build step. Release is a runtime decision.**




**The kill switch is the main benefit.** A bad release you can turn off in thirty seconds is a different kind of incident from one that needs a store review.

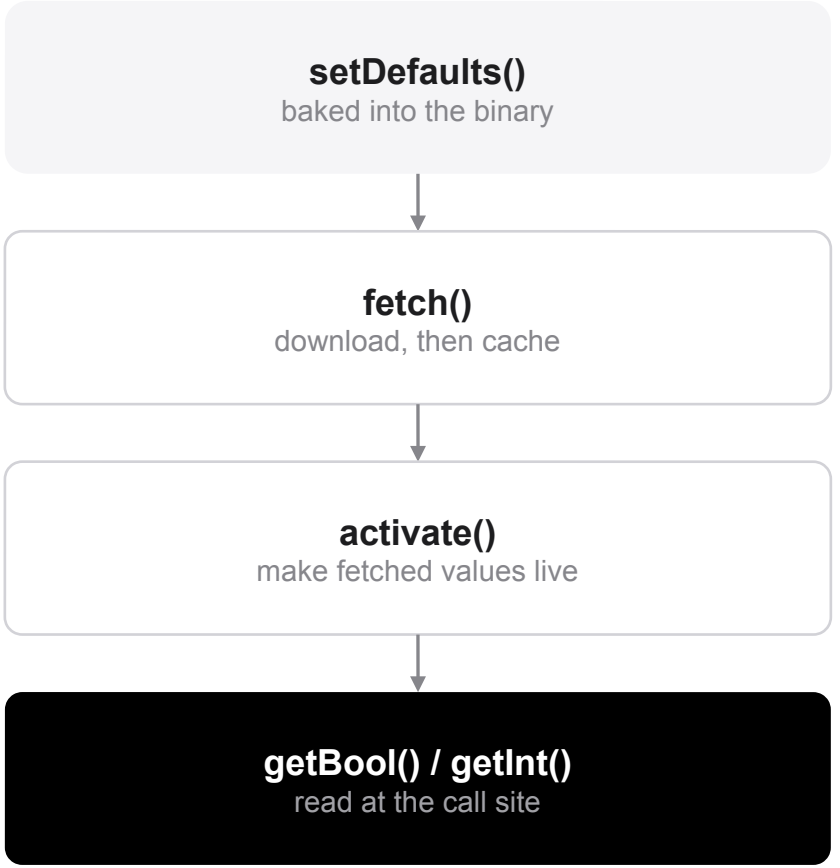
# The tool: Firebase Remote Config

A key/value store hosted by Firebase. Your app ships with in-app defaults, then fetches overrides, so it behaves sensibly on first run, offline, and if the fetch fails.

Values are typed: bool for a flag, int or double for a tuning knob, String or JSON for content you want to change without a release.

Conditions in the console target by app version, platform, language, country, Analytics audience, and by a random percentile, which is how you stage a rollout of behavior rather than of binaries.

Same idea, other tools: LaunchDarkly, Unleash, PostHog, ConfigCat. We use Remote Config because the rest of this lecture is already Firebase.




**Fetch and activate are two steps on purpose.** Values never change underneath a screen that is already rendered.

# Remote Config in Dart

```
final rc = FirebaseRemoteConfig.instance;
// Defaults ship inside the binary: the app works on first run.
await rc.setDefaults(const {
  'checkout_v2_enabled': false,
  'feed_page_size': 20,
});

await rc.setConfigSettings(RemoteConfigSettings(
  fetchTimeout: const Duration(seconds: 10),
  // Production: 1-12 h. Debug: Duration.zero, or you get throttled.
  minimumFetchInterval: const Duration(hours: 1),
));
await rc.fetchAndActivate();   // fetch, then swap the values in

// Read at the call site, every time - never cache it in a global.
if (rc.getBool('checkout_v2_enabled')) return const CheckoutV2();
```

## Rules that keep flags cheap

Fetches are throttled server-side. minimumFetchInterval exists so the backend is not called too often; set it to zero only in debug builds.

Roll out by percentile: 1%, 10%, 50%, 100%, watching crash-free users at each step.

Every live flag is a branch you must keep working. Delete the flag and the dead path as soon as it reaches 100%.

Flags are not security. The disabled code is still in the binary and the values are readable, so keep authorization on the server.

PART 3 · APIS

# Talking to a server

REST, GraphQL, and the cost of each round trip

# REST, and what the verbs promise

Resources named by URLs, acted on with standard HTTP methods; every request carries its own context, so any server can answer it.

| | Safe | Idempotent | What it is for |
|---|---|---|---|
| GET | yes | **yes** | read a resource: /v1/users/42 |
| POST | no | **no** | create; two sends create two things |
| PUT | no | **yes** | replace a resource wholesale |
| PATCH | no | **not in general** | partial update; depends on the payload |
| DELETE | no | **yes** | remove; deleting twice ends the same way |

**Safe and idempotent are two different questions.** Safe means it changes nothing. Idempotent means doing it twice leaves the same state. PUT and DELETE are idempotent but not safe, which is why a client on a flaky mobile network may retry them but must not retry a POST.

# A REST call in Dart

```
import 'dart:convert';
import 'package:http/http.dart' as http;
Future<List<Post>> fetchPosts(String userId) async {
  final uri = Uri.https('api.example.com',
      '/v1/users/$userId/posts', {'limit': '20'});

  final res = await http
      .get(uri, headers: {'Authorization': 'Bearer $token'})
      .timeout(const Duration(seconds: 10));

  if (res.statusCode != 200) {
    throw ApiException(res.statusCode, res.body);
  }
  final json = jsonDecode(res.body) as List<dynamic>;
  return json.map((e) => Post.fromJson(e)).toList();
}
```

## Four things people forget

A timeout. The default on mobile is effectively forever, and a spinner that never ends is a common bug report.

The status code. A 401 or a 500 also has a body, and jsonDecode will happily parse an error page into nonsense.

Codegen. json_serializable + build_runner writes fromJson for you; hand-written parsers break silently when a field is renamed (→ Lecture 5).

dio when you need interceptors, retries, cancellation and one place to refresh an expired token.

# Over-fetching and under-fetching

One profile screen: the user's name, their follower count, and their last 20 posts.

### REST: three round trips

```
GET /v1/users/42
```

returns 38 fields; the screen shows 2

```
GET /v1/users/42/posts?limit=20
```

a second request, after the first returns

```
GET /v1/users/42/followers/count
```

and only now can the screen render

over-fetching · under-fetching · 3 × round-trip time

### GraphQL: one round trip

```
POST /graphql
```

```
user(id: 42) {
  name
  followerCount
  posts(limit: 20) { id title }
}
```

exactly the fields the screen renders · 1 × round-trip time

---

**A round trip on a mobile network costs 50–150 ms, and far more on a bad one.** GraphQL does not make the server faster. It moves the joins server-side, where a careless resolver turns one query into N+1 database calls.

# GraphQL: schema, query, mutation

```
# The schema is the contract: strongly typed and introspectable.
type User {
  id: ID!   name: String!   followerCount: Int!
  posts(limit: Int): [Post!]!
}
# A query reads. The response JSON has exactly this shape.
query ProfileScreen($id: ID!) {
  user(id: $id) {
    name  followerCount
    posts(limit: 20) { id title createdAt }
  }
}

# A mutation writes - and returns the new state, for your cache.
mutation LikePost($postId: ID!) {
  likePost(id: $postId) { id likeCount likedByMe }
}
```

## What the schema buys you

One endpoint, one schema, every client. iOS, Android and web each ask for the fields they render, so there are no per-client endpoints.

Introspection means the tooling is generated, not written: GraphiQL, autocomplete, and Dart types straight from the schema.

! means non-nullable. That maps cleanly onto Dart's null safety, so a nullable field in the schema is a nullable field in your model.

Everything is a POST to one URL, which is why HTTP caching stops helping here.

# GraphQL in a Flutter app

```
// pubspec: graphql_flutter: ^5.2.1
final link = HttpLink('https://api.example.com/graphql');
final client = ValueNotifier(GraphQLClient(
  link: AuthLink(getToken: () async => 'Bearer $token').concat(link),
  cache: GraphQLCache(store: HiveStore()),  // normalized by id
));

Query(
  options: QueryOptions(document: gql(profileScreenQuery),
                        variables: {'id': userId}),
  builder: (result, {fetchMore, refetch}) {
    if (result.isLoading) return const Spinner();
    if (result.hasException) return ErrorView(result.exception!);
    return ProfileView(User.fromJson(result.data!['user']));
  },
);
```

## Two clients, one caveat

**graphql_flutter**

Widget-level Query and Mutation builders, a normalized cache, and dynamic maps. Fastest to start with.

**ferry**

Generates Dart types from your schema and .graphql documents, so a wrong field name is a compile error rather than a runtime null.

GraphQL errors arrive with HTTP 200 and an errors array. Checking the status code is not enough; check result.hasException.

# REST vs GraphQL, side by side

| | REST | GraphQL |
|---|---|---|
| Endpoints | one per resource, many URLs | one URL, usually POST /graphql |
| Payload | fixed by the server; over- and under-fetching | chosen by the client, in one request |
| Caching | HTTP caching, CDNs and ETags, for free | normalized client cache that you configure |
| Schema & typing | OpenAPI, if someone keeps it current | mandatory, introspectable, generates your types |
| Server cost | predictable per endpoint | N+1 resolvers; needs dataloaders and depth limits |
| Errors | carried by the HTTP status code | HTTP 200 plus an errors array |
| Tooling | curl, Postman, any proxy; decades of it | GraphiQL and codegen; harder to read in a proxy |

**Neither one wins.** Simple resources, heavy caching, third-party integrations → REST. Deeply related data, several clients with different needs, expensive round trips → GraphQL. Most real apps end up with both.

PART 4 · OBSERVABILITY

# You cannot debug a phone you do not own

Events → metrics → crashes → alerts

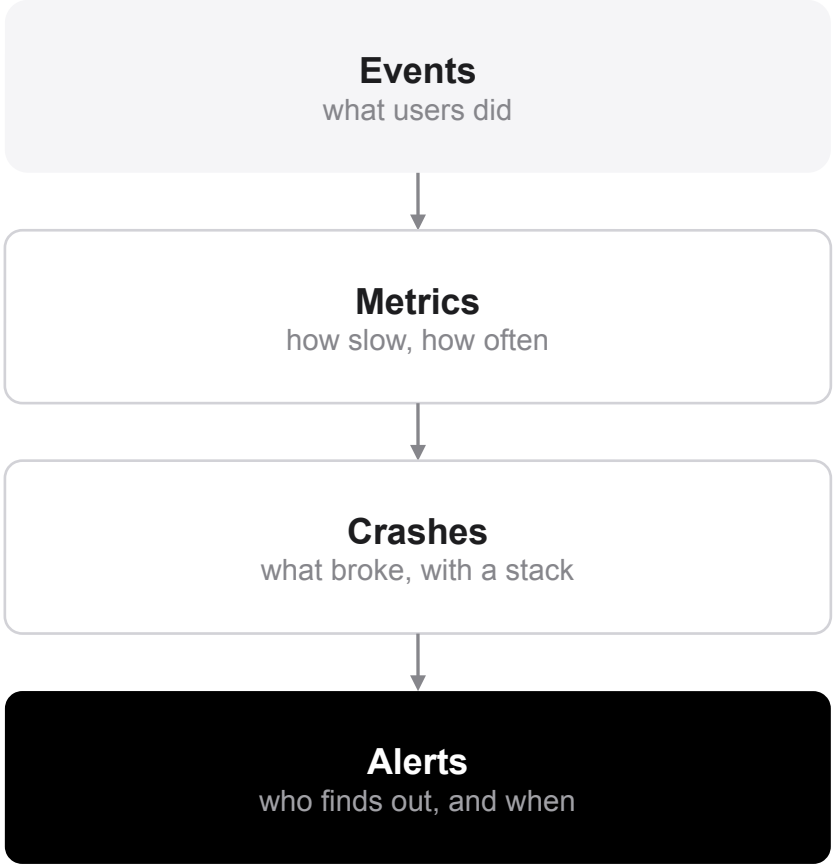
# Why this is harder on a phone

The code runs on hardware you have never seen: thousands of Android models, several OS versions, networks that drop mid-request, devices thermally throttled in a pocket.

You cannot attach a debugger to a user's phone. There is no server log to grep: the log is on the device, and you have no access to it.

Releases are staged and users update slowly, so four versions of your app are live at once and one of them is three months old.

**So the app has to report on itself. The rest of this section is how.**




**Monitoring tells you that something is wrong.** Observability is having enough signal to work out why, without shipping a build to find out.

# Logging an analytics event

```
import 'package:firebase_analytics/firebase_analytics.dart';
final analytics = FirebaseAnalytics.instance;

// snake_case, <= 40 chars; parameter values are String or num.
await analytics.logEvent(
  name: 'checkout_started',
  parameters: {
    'cart_value_ron': 249.90,
    'item_count': 3,
    'payment_method': 'card',
  },
);
// screen_view for free - wire the observer in once.
MaterialApp(
  navigatorObservers: [FirebaseAnalyticsObserver(analytics: analytics)],
);
```

## How to name and use them

first_open, session_start, screen_view and app_remove are collected for you. Everything else is a deliberate decision.

Name events for the funnel you want to read later: checkout_started → payment_submitted → purchase_complete.

Never put personal data in a parameter. No emails, no names and no free text. Analytics is not a log, and this is a GDPR question.

Events are aggregated and delayed by hours. They answer “how many users”, never “what happened to this one user”.

# Catching crashes in Flutter

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
      options: DefaultFirebaseOptions.currentPlatform);
  // Uncaught framework errors -> fatal report + widget stack.
  FlutterError.onError =
      FirebaseCrashlytics.instance.recordFlutterFatalError;

  // Uncaught async errors from outside the framework.
  PlatformDispatcher.instance.onError = (error, stack) {
    FirebaseCrashlytics.instance
        .recordError(error, stack, fatal: true);
    return true;
  };
  runApp(const App());
}
```

## Errors you already caught

```
try {
  await api.pay(cart);
} catch (e, s) {
  await crashlytics.recordError(
      e, s, reason: 'checkout');
}
```

log('card selected') adds breadcrumbs to the next report; setCustomKey and setUserIdentifier add state, for example a hashed id, never an email.

Reports upload on the next launch, so a crash during startup still reaches you.

Verify the wiring once with FirebaseCrashlytics.instance.crash(), then delete that line.

OBSERVABILITY

# Crash-free users: the number to watch

| 99.9% | 99.5% | 1 in 200 |
| --- | --- | --- |
| **crash-free sessions** | **crash-free users** | **users hit a crash** |
| the higher number vendors usually quote | a common floor for a consumer app | at 99.5%; on 400k installs, 2,000 people |

Crash-free users = users with no fatal crash ÷ all users, over a window. It counts people, not events: one user crashing fifty times still costs you one user.

Crash-free sessions is always higher, because most sessions are short and uneventful. Quote users; act on users.

Read it per app version, never in aggregate: a bad release hides inside a healthy overall number for days.

Sort issues by users affected, not by crash count. One issue hitting 4,000 people beats thirty issues hitting two each.

**The metric you act on is crash-free users on the version you are rolling out.** If it drops, halt the rollout or flip the flag.

OBSERVABILITY

# Latency: read p95, not the average

### p50: the median

The typical request. Half your users are faster than this. It is the last number to move when something starts breaking.

### p95

One request in twenty. This is where users start to notice, and where you should set a budget: “feed_load p95 under 1.5 s”.

### p99

The tail: cold starts, retries, 2 G, throttled devices. Real users, but noisy below roughly a thousand samples.

### the average

Almost useless here. One 30-second timeout skews it, and no individual user experiences the average.

**A percentile describes a defined share of your users.** An average describes no particular user.

### Measuring it

```
final t = FirebasePerformance
    .instance.newTrace('feed_load');
await t.start();
await loadFeed();
t.putMetric('items', items.length);
await t.stop();
```

App start (cold, warm and hot) and screen rendering are captured for you as soon as the package is added.

Dart HTTP calls are not: dart:io opens its own sockets and bypasses the instrumented platform stack. Wrap the client, or trace it yourself.

# Which alerts are worth firing

| | Fire when | Why that threshold |
|---|---|---|
| Crash-free users | drops below 99.5% on the version rolling out | one user in 200 is a visible outage, not noise |
| New fatal issue | a signature first seen in the build you shipped | velocity alerts catch regressions, not the old backlog |
| p95 latency | doubles week-over-week on a key endpoint | relative change survives traffic growth; fixed numbers do not |
| Funnel conversion | checkout_started → purchase drops 20% | crash-free can be perfect while the feature is quietly broken |
| Everything else | no alert; put it on a dashboard | an alert nobody acts on trains everyone to ignore all of them |

**Every alert needs an owner and an action.** If the answer to “what would I do about this at 3 a.m.?” is “nothing”, it belongs on a dashboard rather than in an alert.

OBSERVABILITY

# One incident, end to end

Four minutes after a 10% rollout of checkout v2.

14:02

### Analytics

checkout_started is normal, purchase_complete is down 40%, but only on 4.2.0.

→

14:05

### Performance

p95 on the payment trace jumps from 0.4 s to 5.2 s. Something downstream is timing out.

→

14:07

### Crashlytics

A new fatal: null check operator used on a null value, in PaymentResponse.fromJson.

→

14:09

### Remote Config

checkout_v2_enabled → false. Zero users affected. Fix, ship, then re-ramp from 1%.

---

**Events show that something is wrong, metrics show where, crashes show why.** The flag is what limits the damage without waiting for a store review, which is why the two halves of this lecture belong on the same slide.

# Before next week

## Recap

pub add writes the constraint; commit pubspec.lock for an app

Deploy ≠ release: flags give you a percentage rollout and a kill switch

REST and GraphQL trade over-fetching against caching and N+1

Safe ≠ idempotent: it decides what a retry is allowed to do

Events → metrics → crashes → alerts, because you own no device

## This week

Add firebase_analytics and log one event that matters to your project

Wire both Crashlytics handlers, then force a crash and find the report

Put one feature behind a Remote Config bool and flip it live

Write the same profile screen twice: REST calls, then one GraphQL query

## Read more

dart.dev/tools/pub/dependencies

firebase.google.com/docs/remote-config

firebase.google.com/docs/crashlytics/get-started?platform=flutter

graphql.org/learn · pub.dev/packages/graphql_flutter